Further Lean Proof Simplifications

Six additional ideas for `DiophantineConsistency`

Research and implementation report for Vladimir Reshetnikov

26 October 2026

Abstract

This report develops six concrete proof simplifications after auditing the five archives in `docs/incoming` at the initial snapshot, the sixth retired packet linked from its README, and two more packets added during this review. The resulting changes replace two unused internal inductions by an existing zero-or-successor decomposition; reuse an exact Pell parity theorem; compose divisibility statements without opening quotient witnesses; compare Pell indices by monotonicity in the base; keep a nonsquare-norm argument entirely in the integers; and replace an explicit finite-orbit construction by inverse invariance of a finite set.

The accompanying patch preserves the existing public theorem statements. The report distinguishes local proof shortening, helper costs, dependency simplification, and the scope of the Lean checks. The novelty claim is relative to the inspected proposal packets. A promising evaluator refactor was excluded because an earlier note already suggests a closely related semiring witness.

Pinned repository snapshot. `b310142330df89e48413c8fce62e7de223e1c7d5`

Verification versions. Lean 4.32.0; mathlib `81a5d257c8e410db227a6665ed08f64fea08e997`.

Closing audit snapshot. `d163c476db9ca1142d2d73de8fa64c0b2c03a848`. The six target files and dependency pins are unchanged, and the patch also passes its application check at this later snapshot.

Recommended starting point. Apply the two unused-induction deletions, the existing Pell parity theorem, and the integer norm argument first. The Pell base comparison and finite-invariant-set argument are the most useful mathematical changes to the proof structure.

Contents

1 Outcome and measurement 3

2 How existing proposals were excluded 4

3 Idea 1: remove two specialized inductions 5

4 Idea 2: reuse the exact Pell parity theorem 7

5 Idea 3: keep divisibility witnesses abstract 7

6 Idea 4: compare Pell bases instead of exponentials 9

7 Idea 5: prove the norm fact entirely over the integers 10

8 Idea 6: use inverse invariance of a finite set 11

9 Verification and integration 12

10 Further research questions 13

A Source map 15

B Reading the audit artifacts 15

1 Outcome and measurement

The deliverable is a reviewable six-file patch, complete replacement modules, standalone comparison experiments, a prior-proposal exclusion ledger, and validation evidence. The patch changes proofs and imports; it retains the statements of the seven existing public theorems directly edited. The Pell bootstrap file gains one reusable simultaneous monotonicity lemma.

Idea	Main target	Actual simplification
1	Object addition and multiplication at zero	Replace two specialized `omega_ind` scripts and their private formula specifications by `nat_zero_or_succ`.
2	Pell auxiliary-index oddness	Reuse `SignedPellResidue.odd_index_of_base_dvd` with the exact hypothesis already present.
3	Pell sign transport	Apply closure of divisibility under addition, subtraction, and negation instead of constructing four quotient witnesses.
4	Main versus first Pell index	Prove simultaneous monotonicity of the two Pell coordinates in the base, then use strict monotonicity in the index.
5	Nonsquare integer norm	Infer divisibility of square roots and cancel a nonzero integer square; remove the detour through rational squares.
6	Offset-19 exclusion	Use inverse invariance of a finite permutation's forward-invariant even-state set in place of an explicit period and iteration induction.

Module	Before	After	Removed
`ObjectAdditionCancellation`	118	96	22
`ObjectNaturalZero`	60	40	20
`NativePellStepDown`	248	220	28
`NativePellBootstrap`	685	677	8
`MinimumNorm`	308	304	4
`NativeFieldCycles2011`	111	103	8
Total	**1530**	**1440**	**90**

The file totals in this table include comments, whitespace, helper lemmas, and changed imports. They are reproducible source measurements, not a claim about kernel term size or compilation speed. Separate proof-region counts in the validation manifest explain where the savings occur. In particular, the Pell monotonicity helper is charged to the change; the three-line consumer proof is not advertised as a free replacement for the old proof.

The repository's requested Lean tree contains 574 Lean files at this snapshot. The directory named `DiophantineConsistency` contains 391 of those files and 71,601 physical lines, including several check modules. This review selects a small number of proof changes from that larger codebase; it is not a whole-project minimization result.

2 How existing proposals were excluded

2.1 The inspected material

At the initial snapshot, the incoming directory contained these five archives:

1. `Lean_Proof_Simplification_2026-10-26.zip`
2. `ProveIt_Lean_Proof_Simplification_2026-10-26.zip`
3. `diophantine-lean-simplification-2026-10-26.zip`
4. `lean-proof-simplification-2026-10-26 (2).zip`
5. `lean-proof-simplification-2026-10-26.zip`

Their 1,793 archive members contain no nested archives. The downloaded archives' Git blob hashes match the repository directory entries. The audit read their reports and relevant detailed notes and searched the proposed Lean artifacts for overlap. Copies of original source files inside a packet were treated as baseline evidence, rather than automatically treating every theorem in such a copy as a proposal.

The incoming README also links a retired sixth packet, `diophantine-lean-proof-simplification-2026-10-26.zip`, now placed under `research/lean-proof-simplification-20261026`. Its report and three detailed review notes were also inspected. Historical entries concerning other research programs are outside this proof-simplification comparison.

The bundled `audit/exclusion_ledger.json` groups 81 earlier idea families and records 103 checked report or note locators. The readable ledger gives the archive, member path, line, and a distinctive phrase for each entry. The comparison is semantic: changing a proposed abstraction's name is not counted as a new idea.

2.2 Closing refresh after concurrent repository changes

The repository advanced while this review was in progress. At the closing snapshot `d163c476db9ca1142d2d73de8fa64c0b2c03a848`, the original five archives had been placed in research directories, and two new archives were present in `docs/incoming`: `diophantine-lean-simplification-2026-10-26(1).zip` and `lean-proof-simplification-review-2026-10-26.zip`. These were separately audited before delivery. The closing addendum records their hashes, proposal locations, and comparisons with the six retained ideas.

One new packet also proposes an edit in `NativePellStepDown`, but it targets `eq_of_residues` and simplifies casts. That differs from the odd-index theorem reuse and sign-transport changes delivered here. The audit found no matching proposal for any of the six retained mechanisms in the two new packets.

A Git comparison confirms that all six edited source files, the dependency pins, and the imported repository source roots are unchanged between the initial and closing snapshots. The patch passes a separate application check against the later snapshot's index. The main Lean tree acquired additional strip and machine modules; those additions are outside this patch and were not claimed as freshly rebuilt endpoints.

2.3 Topics deliberately left out

Earlier packets already propose assignment naturality, finite witness layouts, scanner iterations and fusion, guarded history folds, exact numerical Boolean outputs, tag dispatch, sparse coefficient bounds, half-radix arithmetic, finite-fiber sum identities, residual-square interfaces, semiring compiler sharing, power-label certificates, indexed DAG equations, shared Split/Gap circuit tails, source/window records, and transport of necessary definable induction. They also contain smaller suggestions in ancillary notes. None of those is offered here as an additional idea.

A concrete evaluator experiment was developed and then excluded. It constructs the semiring evaluator's relational witness and invokes the existing `evalRel_unique` theorem to simplify `ObjectNaturalSemiring.evalRel_iff`. Although that exact consumer refactor is absent from the inspected packets, `notes/syntax.txt:133` in the fifth archive already suggests a semiring witness for evaluator totality. The strict novelty policy used for this delivery excludes that neighboring development from the patch and from the six contributions counted here.

2.4 The novelty boundary of the six retained ideas

The first idea eliminates two inductions whose step cases discard the induction hypothesis. This differs from the earlier plan to transport a necessary induction through a formula specification. The finite-permutation idea uses inverse preservation of a finite state predicate, rather than the earlier scanner-fold and iterate abstractions. The Pell comparison uses an order theorem in the base parameter, rather than a uniform coefficient or exponential envelope. The other three changes are exact local replacements for which the audit found no corresponding proposal.

These are additional contributions relative to the eight inspected packets. The report does not claim that basic mathematical facts such as integer cancellation, monotonicity, or finite-set invariance are new mathematics.

3 Idea 1: remove two specialized inductions

3.1 What the current proofs actually use

`ObjectAdditionCancellation.lean` proves `addRel_zero_iff` by defining a private object-language formula, proving its semantic specification, and applying `omega_ind`. The successor case begins `intro n hn _`: its induction hypothesis is discarded. The same pattern occurs in `ObjectNaturalZero.mulRel_zero_iff`.

The necessary elimination principle is already available: `nat_zero_or_succ` states that every member of a model's internal omega is either its zero or a successor of another member. Importing its existing module and using this decomposition removes the two private formulas and their specification proofs.

3.2 Addition

The relation `AddRel H x y (vempty H)` already supplies the naturalness of its arguments. Apply `nat_zero_or_succ` to the second argument; no new public hypothesis is needed.

If $y = 0$, functionality of addition compares the given graph witness with the zero law and yields $x = 0$. If $y = \mathrm{succ}(n)$, totality supplies the intermediate sum $b = x + n$. The successor law gives $x + y = \mathrm{succ}(b)$. Functionality then forces the empty set to equal that successor, contradicting the fact that b is a member of its successor. No assertion about smaller values of y is used.

```
theorem addRel_zero_iff (H : ZFAxioms mem) (x y : V) :
    AddRel H x y (vempty H) ↔ x = vempty H ∧ y = vempty H := by
  constructor
  · intro hxy
    rcases nat_zero_or_succ H y hxy.2.1 with rfl | ⟨n, hn, rfl⟩
    · exact ⟨(addRel_unique H hxy (addRel_zero H hxy.1)).symm, rfl⟩
    · obtain ⟨b, hb⟩ := addRel_total H hxy.1 hn
      have he : vempty H = vsucc H b := addRel_unique H hxy (addRel_succ H hb)
      exact False.elim (vempty_spec H b (he.symm ▸ vsucc_self H b))
  · rintro ⟨rfl, rfl⟩
    exact addRel_zero H (vempty_in_omega H)
```

3.3 Multiplication

For multiplication, the zero case is immediate. In the successor case, totality gives $b = xn$, and the successor equation identifies $x\,\mathrm{succ}(n)$ with $b + x$. If that sum is zero, the additive zero theorem yields $x = 0$. Again, no induction hypothesis occurs in the argument.

```
theorem mulRel_zero_iff (H : ZFAxioms mem) {x y : V}
    (hx : mem x (omegaV H)) (hy : mem y (omegaV H)) :
    MulRel H x y (vempty H) ↔ x = vempty H ∨ y = vempty H := by
  constructor
  · intro hxy
    rcases nat_zero_or_succ H y hy with rfl | ⟨n, hn, rfl⟩
    · exact Or.inr rfl
    · obtain ⟨b, hb⟩ := mulRel_total H hx hn
      have he : vempty H = addValue H b x := mulRel_unique H hxy (mulRel_succ H hb)
      have hadd : AddRel H b x (vempty H) :=
        he.symm ▸ addValue_rel H (mulRel_result_nat H hb) hx
      exact Or.inl ((ObjectAdditionCancellation.addRel_zero_iff H b x).mp hadd).2
  · rintro (rfl | rfl)
    · exact mulRel_zero_left H hy
    · exact mulRel_zero H hx
```

3.4 Semantic scope and exact savings

The direct decomposition applies to all internal naturals, including nonstandard elements of a model. There is no replacement of internal induction by induction over the external type `Nat`. The existing decomposition theorem itself was proved once by definable omega induction, so the improvement removes two redundant specialized induction scripts rather than removing induction from the foundational dependency chain.

The additive theorem retains its original unguarded statement. The multiplicative theorem retains its two naturalness hypotheses. The changed proof/helper blocks occupy 73 physical lines before the patch and 29 after it. The explicit import of `BoundedZFCConsistency.InternalSat` in each file makes the net full-file reduction 42 lines: 118 → 96 and 60 → 40.

4 Idea 2: reuse the exact Pell parity theorem

4.1 The duplicated argument

In `NativePellStepDown.index_residues`, Pell classification yields an index s with

$$Iu = X_I(s), \qquad I > 1.$$

The original proof then derives oddness of s by a local even-index congruence and divisibility argument. But `SignedPellResidue.odd_index_of_base_dvd` already proves precisely that $I \mid X_I(s)$ forces s to be odd when $I > 1$. The displayed equality supplies its hypothesis directly.

```
have hsodd : Odd s := SignedPellResidue.odd_index_of_base_dvd hI1 (by
  rw [← hIu]
  exact dvd_mul_right I u)
```

The new import is acyclic: `SignedPellResidue` depends on the common Pell integer library, not on `NativePellStepDown`. The reduction also permits removal of the unused local proof that $s > 0$ and an unused name for the second coordinate equality. Public parameters remain in place for compatibility.

4.2 Why this is preferable to another abstraction

No new parity interface or generic congruence framework is needed. This is exact reuse of an existing theorem with the existing hypotheses. The classification step, the odd-index polynomial identity, and the subsequent step-down argument remain explicit. The change reduces the local proof's arithmetic burden and the number of places that must be maintained if the Pell congruence interface changes.

5 Idea 3: keep divisibility witnesses abstract

5.1 The local sign problem

Later in the same `index_residues` theorem, the proof knows

$$c \mid (s - p) \ \vee \ c \mid (s + p), \qquad J \equiv ts \pmod{c}, \qquad t = 1 \ \vee \ t = -1.$$

It must derive $c \mid (J - p)$ or $c \mid (J + p)$. The original proof opens the divisibility statements into integer quotient witnesses in each sign case, constructs a new quotient, and checks its equality by `linear_combination`.

Divisibility already has the required additive closure laws. Convert the congruence once to $c \mid (ts - J)$ and compose it with the known divisibility statement using `dvd_sub`, `dvd_add`, and `dvd_neg`. The branch arithmetic is summarized below.

t	Divisible expressions used	Resulting divisible expression
1	$s - p$, $s - J$	$(s - p) - (s - J) = J - p$
1	$s + p$, $s - J$	$(s + p) - (s - J) = J + p$
-1	$s + p$, $-s - J$	$-((s + p) + (-s - J)) = J - p$
-1	$s - p$, $-s - J$	$-((s - p) + (-s - J)) = J + p$

The delivered proof uses a small local helper and `Or.imp` to map the two branches. It does not introduce a new global "congruence up to sign" relation. The delivered proof uses `convert!` so that the source expression is elaborated without prematurely imposing the target expression. The resulting ring identity then closes the conversion. An explicitly typed intermediate divisibility statement is another readable way to control the same inference.

```
have hJp_of_sign {t : ℤ} (hJs : (J : ℤ) ≡ t * s [ZMOD c])
    (ht : t = 1 ∨ t = -1) :
    (c : ℤ) ∣ (J : ℤ) - p ∨ (c : ℤ) ∣ (J : ℤ) + p := by
  have hJs' := Int.modEq_iff_dvd.1 hJs
  rcases ht with rfl | rfl <;> simp only [one_mul, neg_one_mul] at hJs'
  · exact hsp'.imp
      (fun h => by convert! dvd_sub h hJs' using 1; ring)
      (fun h => by convert! dvd_sub h hJs' using 1; ring)
  · exact hsp'.symm.imp
      (fun h => by convert! dvd_neg.mpr (dvd_add h hJs') using 1; ring)
      (fun h => by convert! dvd_neg.mpr (dvd_add h hJs') using 1; ring)
```

5.2 When this technique is useful

Opening an existential divisibility witness is appropriate when the quotient itself becomes later data. Here the quotient is immediately repackaged into another proposition. Leaving it abstract makes the proof expose its actual structure: an additive combination of elements of the principal ideal $c\mathbb{Z}$. The argument retains both possible signs and adds no positivity condition on the signed intermediate coordinate.

Ideas 2 and 3 affect separate regions of one theorem. Their net source savings are reported together at the file level, so the two changes are not counted twice.

The adjacent `signed_index_residues` wrapper receives the same treatment. After the two cases of `Int.natAbs_eq`, it reuses the supplied congruence `hVJ` or its negation directly. This reduces the absolute-value congruence block from 11 to five lines. The identity for its square is obtained from `Int.natCast_natAbs` and `sq_abs`; the argument never equates a negative integer with its absolute value. These are supporting edits within the sign-transport idea, not additional contributions.

6 Idea 4: compare Pell bases instead of exponentials

6.1 The current comparison

The original `NativePellBootstrap.main_index_gt_first` assumes $1 < A$, $1 < P$, and

$$c = Y_A(p), \qquad k = 2Y_P(n), \qquad k < c.$$

It also assumes $c > 1$, $p > 0$, $n > 0$, and the relatively strong separation $2A < 2P - 1$. Its 21-line proof first establishes lower bounds on the indices, then compares

$$c \le (2A)^{p-1} \le (2A)^{n-1} < (2P-1)^{n-1} \le Y_P(n) \le k$$

under the contradictory assumption $p \le n$.

The desired conclusion follows from a more direct order comparison.

6.2 Simultaneous monotonicity

For a natural parameter $a > 1$, write $X_a(n)$ and $Y_a(n)$ for the two Pell coordinates. Their first-order recurrences are

$$\begin{aligned} X_a(0) &= 1, & Y_a(0) &= 0, \\ X_a(n+1) &= aX_a(n) + (a^2-1)Y_a(n), & Y_a(n+1) &= X_a(n) + aY_a(n). \end{aligned}$$

All coefficients are nonnegative. If $1 < a \le b$, simultaneous induction on n gives

$$X_a(n) \le X_b(n) \quad \text{and} \quad Y_a(n) \le Y_b(n).$$

The simultaneous statement is the useful induction invariant: trying to prove the inequality for the second coordinate alone would need information about the first coordinate in the successor case.

```
theorem coordinates_mono_base {A B : ℕ} (hA : 1 < A) (hB : 1 < B)
    (hAB : A ≤ B) (n : ℕ) :
    xn hA n ≤ xn hB n ∧ yn hA n ≤ yn hB n := by
  induction n with
  | zero => simp
  | succ n ih =>
    exact ⟨Nat.add_le_add (Nat.mul_le_mul ih.1 hAB)
      (Nat.mul_le_mul (Nat.sub_le_sub_right (Nat.mul_le_mul hAB hAB) 1) ih.2),
      Nat.add_le_add ih.1 (Nat.mul_le_mul ih.2 hAB)⟩
```

The proof uses the positive first-order recurrence, avoiding the subtraction that occurs in an alternative second-order Pell recurrence. Natural subtraction in $a^2 - 1$ is handled by its ordinary monotonicity theorem.

Proposition 6.1. *Let $1 < A \le P$. If $Y_P(n) < Y_A(p)$, then $n < p$.*

Proof. Base monotonicity gives $Y_A(p) \le Y_P(p)$. Hence $Y_P(n) < Y_P(p)$. Strict monotonicity of the second coordinate in its index then gives $n < p$. □

Since $Y_P(n) \le 2Y_P(n) = k < c = Y_A(p)$, the proposition proves the target. The compatibility theorem keeps the old hypotheses but now has a three-line body:

```
theorem main_index_gt_first {A P c k p n : ℕ}
    (hA : 1 < A) (hP : 1 < P) (hc : 1 < c)
    (hp : 0 < p) (hn : 0 < n) (hAP : 2 * A < 2 * P - 1)
    (hcp : c = yn hA p) (hk : k = 2 * yn hP n) (hkc : k < c) : n < p := by
  have hmono := (coordinates_mono_base hA hP (by omega : A ≤ P) p).2
  rw [hcp, hk] at hkc
  exact (strictMono_y hP).lt_iff_lt.mp (by omega)
```

6.3 A useful strengthening, with its cost made explicit

The standalone experiment also checks the stronger theorem whose assumptions are only $1 < A$, $1 < P$, $A \le P$, the two coordinate identities, and $k < c$. The old hypotheses $c > 1$, $p > 0$, $n > 0$, and strict base separation are not needed for this logical implication. Existing callers retain their public interface in the integration patch.

The old consumer proof has 21 body lines and the replacement has three. The new monotonicity helper occupies 11 physical lines including its declaration and documentation, and a separating blank line is retained. Thus this is a small net reduction even when the reusable helper is charged immediately. More significantly, it removes four exponential comparison steps, truncated index subtraction, and two positivity subarguments from the consumer.

The current source still proves the stronger base-separation fact for its other uses. Replacing that theorem, changing its public statement, or deleting its proof is not part of this patch. A future cleanup can inspect its complete caller set and decide whether the weaker base inequality is sufficient there.

7 Idea 5: prove the norm fact entirely over the integers

7.1 The source argument and its replacement

`MinimumNorm.norm_ne_zero_of_not_isSquare` asserts

$$\neg\,\mathrm{IsSquare}(d), \quad y \neq 0 \quad \Longrightarrow \quad x^2 - dy^2 \neq 0 \qquad (d, x, y \in \mathbb{Z}).$$

The original proof casts the equality into $\mathbb{Q}$, introduces x/y, clears a denominator, and transfers rational squareness back to the integers.

An integral proof is direct. Suppose $x^2 = dy^2 = y^2 d$. Then $y^2 \mid x^2$. The integer theorem `Int.pow_dvd_pow_iff` gives $y \mid x$, so write $x = yk$. Substitution yields

$$y^2 k^2 = y^2 d.$$

Because $y \neq 0$, its square is nonzero and can be cancelled. Thus $d = k^2$, contradicting the hypothesis.

```
theorem norm_ne_zero_of_not_isSquare (d x y : ℤ) (hd : ¬ IsSquare d) (hy : y ≠ 0) :
    x ^ 2 - d * y ^ 2 ≠ 0 := by
  intro h
  have he : x ^ 2 = y ^ 2 * d := by nlinarith only [h]
  obtain ⟨k, hk⟩ := (Int.pow_dvd_pow_iff two_ne_zero).mp
    (show y ^ 2 ∣ x ^ 2 from ⟨d, he⟩)
  exact hd ⟨k, by
    rw [hk, mul_pow] at he
    simpa only [pow_two] using (mul_left_cancel₀ (pow_ne_zero 2 hy) he).symm⟩
```

7.2 Why the coefficient domain matters

The decisive statement is the divisibility-of-powers theorem for integers. This should not be generalized silently to an arbitrary integral domain: in a domain that is not integrally closed, an element can acquire a square root in its fraction field without having one in the domain. The delivered proof keeps the original integer statement and its hypothesis $y \neq 0$.

The full replacement file compiles after removing the direct imports of `Mathlib.Data.Rat.Lemmas` and `Mathlib.Tactic.FieldSimp`. This is a reduction in the module's direct dependency vocabulary. It does not prove that no transitive dependency elsewhere in the import graph mentions rational numbers, and no compilation-time improvement is claimed.

8 Idea 6: use inverse invariance of a finite set

8.1 The finite-state proof obligation

`NativeFieldCycles2011.offset19_excludes_510` works with a permutation σ of a finite type, a state value $S_i < 512$, and a bit $\epsilon_i \leq 1$, satisfying

$$S_{\sigma i} = (2S_i + \epsilon_i) \bmod 512.$$

The exceptional bit may equal one only at a single fixed source value s_*. The existing predecessor argument shows that if some value were 510, then $s_* = 511$. The existing local successor lemma then shows

$$S_i \equiv 0 \pmod 2 \quad \Longrightarrow \quad S_{\sigma i} \equiv 0 \pmod 4.$$

The original final step proves evenness along all forward iterates, obtains a positive period, constructs its predecessor index $p - 1$, and uses return to the starting point to get the contradiction $510 \equiv 0 \pmod 4$.

8.2 The shorter proof

Let $E = \{i \mid S_i \equiv 0 \pmod 2\}$. The successor lemma implies $\sigma(E) \subseteq E$. The restriction of an injective map to the finite set E is surjective, so $\sigma^{-1}(E) \subseteq E$ as well. Consequently, if $S_i = 510$, the predecessor $\sigma^{-1}i$ is even. Apply the same successor lemma to that predecessor. Its successor is i, so S_i is divisible by four, which contradicts $510 \bmod 4 = 2$.

This exact finite-permutation fact is already present in mathlib as `Equiv.Perm.perm_symm_on_of_perm_on_finite`. The new final part of the proof is:

```
have heven : ∀ j, S j % 2 = 0 → S (sigma j) % 2 = 0 := by
  intro j hj
  have hm := successor_mod_four sigma S epsilon he hstep hgate' j hj
  omega
have hpre : S (sigma.symm i) % 2 = 0 :=
  Equiv.Perm.perm_symm_on_of_perm_on_finite heven (by omega)
have hm := successor_mod_four sigma S epsilon he hstep hgate' _ hpre
simp only [Equiv.apply_symm_apply, hi] at hm
omega
```

The direct import changes from the periodic-point library to `Mathlib.GroupTheory.Perm.Finite`. The local arithmetic lemmas and the separate offset-20 theorem remain the same. The full-file reduction is eight physical lines; the larger benefit is removal of the custom orbit induction, period witness, and predecessor-index arithmetic.

8.3 What cannot be dropped

Finiteness is essential to this inference. On an infinite orbit indexed by $\mathbb{Z}$, a bounded state can stay at 511 for all negative times, take the zero exceptional bit once, and then evolve through

$$510, 508, 504, 496, 480, 448, 384, 256, 0, 0, \ldots.$$

Use the exceptional bit one while remaining at 511, and zero from the transition to 510 onward. This satisfies the recurrence and the fixed-source gate with $s_* = 511$ on the infinite shift permutation. Thus neither this argument nor the original one licenses deleting the finiteness hypothesis.

The permitted finite 511 self-loop also remains permitted. The theorem excludes the value 510, not all nonzero cycles.

9 Verification and integration

All six complete replacement modules compiled successfully together in one combined module tree with Lean 4.32.0 and the exact pinned mathlib revision. The unchanged repository dependencies were built from the pinned sources; the exact-version mathlib cache was reused. The combined run loaded the new addition proof when checking the multiplication file and the new step-down module when checking the Pell bootstrap file.

The existing `NativeFieldCycles2011Checks` module also compiled against the replacement. Its checks include the allowed nonzero 511 self-loop, the zero loop, use of the offset-19 exclusion, and the offset-20 statement on an infinite type with an arbitrary map.

The selected axiom audit reports only `propext`, `Classical.choice`, and `Quot.sound` for the seven edited existing theorems. The new `coordinates_mono_base` lemma reports only `propext`. No `sorryAx`, native-decision trust axiom, or new axiom appears in those results. Separate experiments also checked the stronger Pell comparison and the direct internal-natural proofs.

The combined patch passes `git apply --check` at the pinned commit. The seven retained public declaration headers match the originals after whitespace normalization. The compiler leaves only the expected warnings about public parameters retained for compatibility: `hy` in the step-down theorem and `hp`/`hn` in the index comparison.

These checks cover the six changed modules and the stated regression and axiom checks. The entire ProveIt repository and every downstream endpoint were not rebuilt. The two removed direct imports in `MinimumNorm` have no proof-bearing direct consumer in this directory beyond the import and axiom-audit aggregators inspected during the review.

9.1 Reproducibility

The patch is pinned to the repository commit on the title page. It is intended to be applied at the repository root. The archive contains:

- the LaTeX source and this PDF;
- one combined patch and complete replacement files;
- independent Lean experiments for the new mathematical arguments;
- a verification script using the repository's pinned Lean/Lake setup;
- source hashes, source-line metrics, successful compiler logs, and axiom output;
- the exclusion ledger, candidate novelty comparison, and independent review notes.

The verification script applies no patch itself. It checks the expected replacement hashes in an already patched checkout, builds the six affected modules, and runs the axiom audit. This keeps the review and mutation steps explicit. A preparatory `git apply --check` command is documented in the README before the ordinary `git apply` command.

9.2 What the evidence supports

The checked replacement modules demonstrate that their retained declarations elaborate and their proofs are accepted with the pinned dependencies. The standalone experiments make it easy to inspect the essential arguments without the surrounding source file. The axiom report checks the selected results for admitted proofs or extra logical assumptions.

Line counts are deterministic textual measurements. They do not measure the size of a kernel proof term, the transitive size of imports, or the time saved on a cold project build. Compilation times in local logs are diagnostic only: the environment reused an exact-version package cache and was not a controlled benchmarking setup.

10 Further research questions

10.1 Where should Pell parameter monotonicity live?

The patch places the new helper immediately above its first consumer for a small, reviewable change. If other modules need it, the natural destination is a lower Pell sequence interface, with

public names for the two projections of the simultaneous theorem. A follow-up can inventory genuine consumers before adding more wrappers. It should compare the complete helper cost with the proof reductions at those call sites.

10.2 Can the base-separation obligations be weakened downstream?

The strengthened standalone theorem needs only $A \leq P$. Inspecting all uses of `source_base_separation` may reveal callers whose strict separation proofs are now unnecessary. This requires an actual dependency and caller audit; this report does not claim that the existing separation theorem can already be deleted. The present patch preserves its public interface.

10.3 Which internal inductions have no inductive content?

The two successful examples suggest a targeted audit of `omega_ind` uses whose step binder is literally discarded. The test is semantic: a proof may hide an induction hypothesis in automation or use it under a different name. Where only zero/successor elimination is needed, the existing internal decomposition theorem should replace the formula-specific induction. Necessary internal inductions must retain their definability witnesses; unrestricted induction over arbitrary external predicates would change the mathematics.

10.4 Can signed divisibility endpoints share a small algebraic lemma?

The delivered patch uses a local helper because there is one identified consumer. If a search finds several independent endpoints with the same $\pm$-transport pattern, a public theorem about divisibility up to sign could make them shorter. Its statement should encode the orientation of `Int.ModEq` explicitly and preserve all sign possibilities. A global API is warranted only after that reuse is demonstrated.

10.5 Can other finite-state proofs use predicate invariance?

Some finite-permutation arguments need an actual period or orbit length; those should keep the periodic-point interface. When the only use of periodicity is to return a forward-invariant property to a predecessor, the finite-set lemma is a better fit. The relevant search should inspect the proof's consumer of the period witness, not replace every finite orbit argument mechanically.

10.6 How should proof simplification be scored?

A useful follow-up benchmark can record four separate quantities: edited source lines including helpers, proof-term size, import footprint, and elaboration time under controlled conditions. A proof can improve one without improving all four. The present work establishes source and structural simplifications, with formal checking of the edited modules, and leaves performance claims to such a benchmark.

A Source map

All repository links below resolve to the pinned snapshot. The exact baseline and replacement file hashes are in `validation/manifest.json`.

Source	Role in this report
`ObjectAdditionCancellation.lean`	Original zero-sum induction and retained public statement.
`ObjectNaturalZero.lean`	Original zero-product induction and its downstream instance.
`InternalSat.lean`	Internal zero-or-successor decomposition; no standardness assumption.
`SignedPellResidue.lean`	Existing exact odd-index theorem.
`NativePellStepDown.lean`	Auxiliary parity and signed divisibility transport.
`NativePellBootstrap.lean`	Original exponential comparison of the main and first indices.
`Diophantine/Common/Pell.lean`	Pell recurrence and monotonicity interface used by the new base comparison.
`MinimumNorm.lean`	Original rational-square norm argument.
`NativeFieldCycles2011.lean`	Existing predecessor and successor arithmetic for the offset exclusion.
`Mathlib/GroupTheory/Perm/Finite.lean`	Finite forward-invariant sets are invariant under the inverse permutation.
`docs/incoming/README.md`	Current and retired proposal packet inventory.

B Reading the audit artifacts

The exclusion ledger is intentionally more detailed than the prose summary. An entry records an earlier idea family, its targets, and one or more precise locations in archived material. It is a reference for overlap screening, not an additional list of proposed work. The novelty comparison marks the canonical-evaluation candidate as excluded and the unused constructor-algebra design as not delivered.

The independent review notes examine logical scope: preservation of nonstandard internal naturals, dependence on finite permutations, retention of the 511 self-loop, sign orientations, and the absence of circular use of the theorems being replaced. Compiler acceptance and this conceptual review serve different purposes; both are included so the intended simplification is easy to assess.